Hong Kong University of Science and Technology
COMP 4211: Machine Learning
Fall 2025

Programming Assignment 2 (CRNN OCR)
Due: 7 November 2025, Friday, 11:59pm

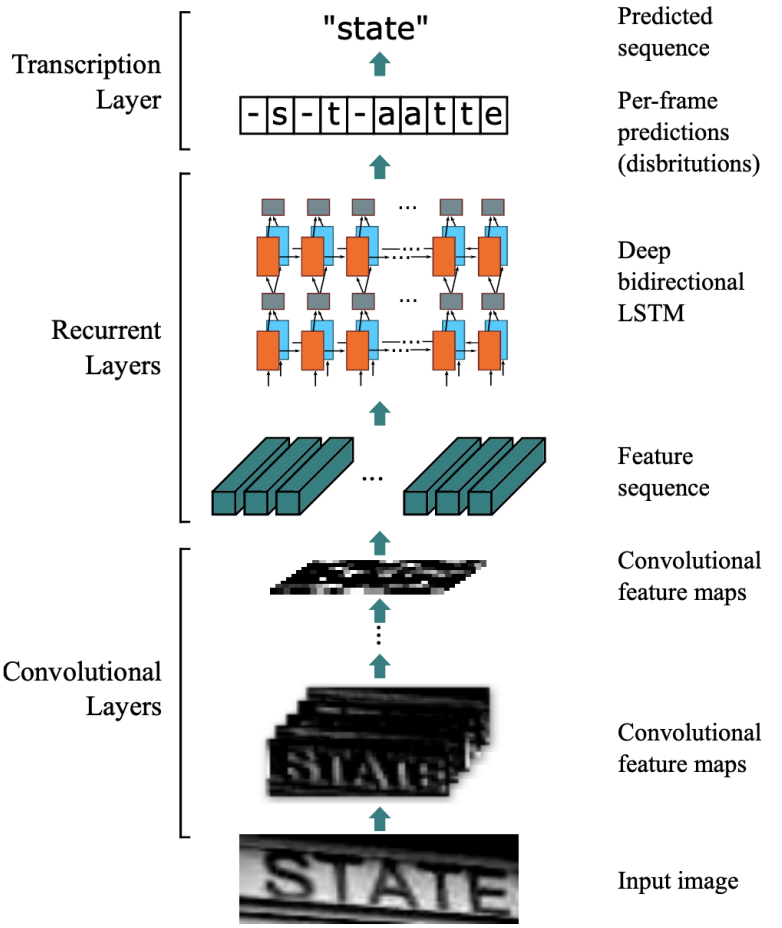

Figure 1: CRNN pipeline.

1 Introduction

This assignment gives you hands-on experience implementing a **CRNN** (Convolutional Recurrent Neural Network) for scene text recognition. You will (1) build a CNN+BiLSTM model with **CTC loss** for sequence transduction, (2) implement data pipelines for variable-width text images, (3) evaluate using **Accuracy**, and (4) perform clustering.

2 Objectives

By the end, you should be able to:

- Implement a CRNN with TensorFlow/Keras for OCR.
- Prepare datasets with label encoding/decoding and compute `(input_length, label_length)`

for CTC.
- Train with `tf.GradientTape` and `tf.nn.ctc_loss` / `tf.keras.backend.ctc_batch_cost`.
- Decode with greedy or beam search; report **Accuracy**.
- Perform clustering on vectors.

3 Notation for Tasks

Use the markers $[C_n]$ for code and $[Q_n]$ for written answers in the markdown cells.

4 Environment Setup

We recommend Google Colab with GPU.

- `tensorflow==2.19.0`, `cuda==12.2` (tested)

5 Part 1: Convolutional Recurrent Neural Network (CRNN)

5.1 Dataset

Download the dataset and split it into training, validation and test sets using the code provided.

[C_1] Implement the `Synth90kDataset` class. Fill in the missing parts as follows. Define the class variables and implement methods:

1. `CHARS` (string): the character set with this order: `0 - 9`, `a - z`, followed by `A - Z`.
2. `CHAR2LABEL` (dict): the mapping from the characters in `CHARS` to the indices (starting from 1).
3. `LABEL2CHAR` (dict): the mapping from the indices (starting from 1) to the characters in `CHARS`.
4. Implement the `_load_from_raw_files` method.
5. Implement the `_process_single_item` method.
6. Implement the `create_tf_dataset` method that creates a TensorFlow Dataset suitable for both training and validation. Your implementation should:
   - Use a generator that yields processed items using the helper method `_process_single_item`
   - Handle image and text data with variable-length sequences
   - Apply **padded batching** to handle the variable-length text sequences
   - Support both training and validation modes with proper batch handling

   Your implementation should respect the following requirements:
   - Use `padded_batch` with appropriate padding values
   - Only drop incomplete batches during training
   - Apply prefetching for training
   - Maintain the correct output signature for the data

   Hints:
   - You may make use of the helper method `_process_single_item`
   - Consider using `tf.data.Dataset.from_generator`
   - Use the provided constants like `self.img_height`, `self.img_width`, etc.
   - If your implementation is correct, the `element_spec` will return a tuple as follows:

```
(TensorSpec(shape=(<batch_size>, <img_height>, <img_width>, 1),
    dtype=tf.float32, name=None),
```

```
TensorSpec(shape=(<batch_size>, <max_label_len>),
   dtype=tf.int32, name=None),
TensorSpec(shape=(<batch_size>,),
   dtype=tf.int32, name=None))
```

5.2 Build the CRNN model

Component	Description	Output Shape
`cnn`	CNN Backbone (described below)	(`None`, H/16 - 1, W/4 - 1, 512)
Feature Reshape	Reshape for sequence processing	(batch, W/4 - 1, features)
Map to Sequence	Dense layer: map_to_seq_hidden units	(batch, W/4 - 1, map_to_seq_hidden)
RNN Layers	2× Bidirectional LSTM, rnn_hidden units each	(batch, W/4 - 1, 2×rnn_hidden)
Output	Dense layer: num_class units + Transpose	(W/4 - 1, batch, num_class)

Table 1: CRNN model architecture overview, where its input shape is (H, W, channel)

Implement the `_build_cnn_backbone` method in the `CRNN` class. The CNN architecture is stated in Table 2:

Layer	Type	Configuration	Output Shape
0	Conv2D	Filters: 64, Kernel: 3x3, Stride: 1, Padding: 'same'	(H, W, 64)
	Activation	ReLU/ LeakyReLU(alpha = 0.2)	
	MaxPooling2D	Pool: 2x2, Stride: (2, 2)	(H/2, W/2, 64)
1	Conv2D	Filters: 128, Kernel: 3x3, Stride: 1, Padding: 'same'	(H/2, W/2, 128)
	Activation	ReLU/ LeakyReLU(alpha = 0.2)	
	MaxPooling2D	Pool: 2x2, Stride: (2, 2)	(H/4, W/4, 128)
2	Conv2D	Filters: 256, Kernel: 3x3, Stride: 1, Padding: 'same'	(H/4, W/4, 256)
	Activation	ReLU/ LeakyReLU(alpha = 0.2)	
3	Conv2D	Filters: 256, Kernel: 3x3, Stride: 1, Padding: 'same'	(H/4, W/4, 256)
	Activation	ReLU/ LeakyReLU(alpha = 0.2)	
	MaxPooling2D	Pool: 2x1, Stride: (2, 1)	(H/8, W/4, 256)
4	Conv2D	Filters: 512, Kernel: 3x3, Stride: 1, Padding: 'same'	(H/8, W/4, 512)
	BatchNorm		
	Activation	ReLU/ LeakyReLU(alpha = 0.2)	
5	Conv2D	Filters: 512, Kernel: 3x3, Stride: 1, Padding: 'same'	(H/8, W/4, 512)
	BatchNorm		
	Activation	ReLU/ LeakyReLU(alpha = 0.2)	
	MaxPooling2D	Pool: 2x1, Stride: (2, 1)	(H/16, W/4, 512)
6	Conv2D	Filters: 512, Kernel: 2x2, Stride: 1, Padding: 'valid'	(H/16 - 1, W/4 - 1, 512)
	Activation	ReLU/ LeakyReLU(alpha = 0.2)	

Table 2: Architecture of the CNN backbone in the CRNN model. Input shape is (H, W, channel). ReLU/ LeakyReLU means it uses LeakyReLU if `leaky_relu` is True, otherwise it uses ReLU.

[Q_1] Referring to Table 2, please calculate the total number of trainable parameters in `Conv2D` of layer 4. Show your steps.

[Q_2] Referring to Table 2, please explain the output shape of layer 6.

[Q_3] Referring to Table 2, the `MaxPooling2D` in layer 3 and layer 5 are designed to be `stride = (2, 1)` instead of `stride = (2, 2)`. By considering the architecture of `CRNN`, please explain the purpose of that.

[C_2] Implement the `__init__`, `call`, and `get_sequence_representation` methods in the `CRNN` class according to Table 1.

[Q_4] The RNN part in `CRNN` processes the sequence in the width dimension of the output of the CNN part. Explain:

- How each position in the sequence (each column of features) corresponds to different regions in the original image. Please explain it by the ratio of the original image width to the sequence length.
- Why this approach is effective for recognizing text of variable lengths

[Q_5] The model uses two stacked bidirectional LSTM layers. Discuss:

- What hierarchical features does each LSTM layer learn?
- The trade-offs between using more vs. fewer LSTM layers. You might add/remove the LSTM layers for experiments.

5.3 Connectionist Temporal Classification Loss (CTC Loss)

The connectionist temporal classification has the following definition. Given:

- Input sequence: $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\}$ where $\mathbf{x}_t \in \mathbb{R}^D$
- Target sequence: $\mathbf{y} = \{y_1, y_2, \ldots, y_U\}$ where $y_i \in \mathcal{L}$
- Extended alphabet: $\mathcal{L}' = \mathcal{L} \cup \{\epsilon\}$ where ϵ is the blank symbol

The CTC loss is defined as:

$$\mathcal{L}_{CTC} = -\log P(\mathbf{y} \mid \mathbf{X}) = -\log \left(\sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} P(\pi_t \mid \mathbf{x}_t) \right)$$

where:

- $\pi = (\pi_1, \pi_2, \ldots, \pi_T)$ is an alignment path with $\pi_t \in \mathcal{L}'$
- $\mathcal{B} : \mathcal{L}'^T \to \mathcal{L}^{\leq T}$ is the mapping that removes repeated characters and blank symbols
- $P(\pi_t \mid \mathbf{x}_t) = \text{softmax}(\texttt{logit})_{\pi_t}$

[C_3] Implement the `CTCLoss` class. You might use `tf.nn.ctc_loss` and `tf.reduce_mean`.

[Q_6] Why can't we use the standard cross-entropy loss for the CRNN text recognition task? What specific challenges does CTC solve that cross-entropy cannot handle? You might give an

example for explanation.

5.4 Connectionist Temporal Classification Decoding (CTC Decoding)

The `CRNN` outputs a matrix of probabilities over T time-steps and C classes. CTC decoders are to find the most likely output sequence, considering that many different alignments can collapse into the same final sequence by removing repeated characters and `blank` tokens. There are three types of CTC decoders:

- **Greedy Decoder**, your task
  At each time-step, it picks the character with the highest probability. Then, it forms the final output by collapsing this sequence: removing all blank tokens and merging repeated characters

- **Beam Search Decoder**, which has been implemented for you :)
  It keeps track of the top `beam_size` candidate sequences (beams) at each time step. For each candidate, it explores extensions with every possible character, calculates the new cumulative score (sum of log probabilities), and only keeps the best `beam_size` results. After processing all time-steps, it finds the most probable sequence by collapsing all candidates in the final beam and summing probabilities for those resulting in the same label sequence.

- **Prefix Beam Search Decoder**, your optional bonus task
  An optimized version of beam search. Instead of storing full paths, it aggregates probabilities for prefixes (partially decoded sequences). It cleverly tracks two separate scores for each prefix: the probability of paths ending with a blank (lp_b), and the probability of paths ending with a non-blank character (lp_nb). This avoids the redundancy of storing many paths that will collapse to the same prefix

[C_4] Implement the function `greedy_decode`.

[B_1] Implement the function `prefix_beam_decode`.

[Q_7] Compare the three decoders in terms of performance and computational costs.

5.5 Train the model

Implement the marked `TODO`s in the provided trainer skeletons. The training loop orchestration is already implemented for you in `BaseTrainer.fit()`; do *not* rewrite a separate loop. Instead, complete the CRNN-specific pieces as instructed below.

[C_5] **Complete the `CRNNTrainer` `TODO`s:**

a) `build_datasets()`: return training/validation `tf.data.Datasets` yielding (images, targets, target_lengths) with shapes/dtypes exactly as documented in the file.

b) `build_model()`: instantiate `CRNN` with correct `num_class` and hyperparameters; run a one-batch sanity pass and print shapes; optionally load weights if `reload_checkpoint` is set.

c) `build_optimizer()`: return an Adam optimizer (use LR from config).

d) `build_loss()`: return your `CTCLoss` instance/wrapper.

e) `forward(images, training)`: simple model forward; keep batch-major $[B, T, C]$.

f) `compute_loss(logits, batch)`: call the CTC criterion.

g) `@tf.function train_step(batch)`: run forward, compute loss, `tape.gradient(...)` and `optimizer.apply_gradients(...)`; return the scalar loss.

5.6 Evaluation

Implement the marked `TODO`-block inside the provided evaluation helper and use it from the trainer.

[C_6] In the function `evaluate(model, dataloader, criterion, ...)`:

a) **Forward:** run the model on each batch to obtain logits.

b) **Loss:** compute CTC loss with the given `criterion`. If your loss expects time-major logits, transpose to $[T, B, C]$ before the call.

c) **Decode:** use `ctc_decode(...)` with the configured method (`greedy` or `beam_search`) and beam size.

d) **Compare:** collapse predictions, compare with targets using exact match, and collect wrong cases.

e) **Accumulate:** update running totals for loss, count, and accuracy; respect `max_iter` for early stopping; advance the progress bar each iteration.

Call this `evaluate(...)` from `CRNNTrainer.evaluate_model()` so that validation runs automatically at the configured interval.

[Q_8] Train the model for at least 4 epochs. Report your test metrics and show 6 error cases (image, GT, pred, edit ops).

[C_7] In `train_step`, add gradient clipping by norm. Clips the tensor values to a maximum L2-norm of 5. Hint: you might use `tf.clip_by_norm`.

[Q_9] Compare the stability of the model training with/without gradient clipping. Briefly explain.

6 Part 2: Clustering

6.1 Setup and Notation

Let the CRNN produce, for each test image, a sequence of pre-softmax logits

$$\mathbf{Z} = \{\mathbf{z}_1, \ldots, \mathbf{z}_T\}, \qquad \mathbf{z}_t \in \mathbb{R}^C,$$

where T is the number of time-steps (width after the CNN backbone), and $C = 63$ is the number of classes (62 characters + blank). Define the per-step softmax

$$\mathbf{p}_t = \mathrm{softmax}(\mathbf{z}_t), \qquad \hat{y}_t \;=\; \arg\max_{c \in \{0,\ldots,62\}} \big(\mathbf{p}_t[c]\big),$$

with $\hat{y}_t = 0$ denoting the blank.

Let there be N test images; stack all time steps from all test images into a single matrix of logits

$$\mathbf{Z}_{\text{flat}} \in \mathbb{R}^{M \times C}, \quad M = \sum_{i=1}^{N} T_i \text{ (equivalently } M = N \cdot \overline{T} \text{ if uniform).}$$

Let $\hat{\mathbf{y}}_{\text{flat}} \in \{0, \ldots, 62\}^M$ be the corresponding greedy labels, obtained by concatenating all $\hat{y}_t$.

Note: You may consider saving the $\mathbf{Z}_{\text{flat}}$ as a .npy file so that you can reuse the vectors without running the model in the future. You can also use scikit-learn library.

6.2 Questions

[C_8] **Logits Extraction on Test Set.**

a) Run the trained CRNN *only* on the test split to obtain per-step logits $\mathbf{Z}$ for each image; do *not* apply softmax before clustering.

b) Concatenate into $\mathbf{Z}_{\text{flat}} \in \mathbb{R}^{M \times C}$.

c) Compute greedy per-step labels $\hat{\mathbf{y}}_{\text{flat}}$ by arg max over the softmax of each row.

[C_9] **Standardization and Clustering on Logits.**

a) Standardize features column-wise: $\widetilde{\mathbf{Z}} = \text{Standardize}(\mathbf{Z}_{\text{flat}})$.

b) Perform clustering on $\widetilde{\mathbf{Z}}$ using KMeans(K), where $K = 63$ (You could also choose to do Agglomerative).

[C_{10}][Q_{10}] **Silhouette and Purity on Test Logits.**

a) **Silhouette Score.** Report the Silhouette

$$\text{Sil} = \frac{1}{M} \sum_{m=1}^{M} \frac{b(m) - a(m)}{\max\{a(m),\, b(m)\}},$$

where $a(m)$ is the mean intra-cluster distance for point m, and $b(m)$ is the smallest mean distance from m to a different cluster. If M is large, you may compute Sil on a uniform random subset of size 50,000.

b) **Cluster Purity vs. Greedy Labels.** Let $\mathcal{C}_k$ denote the index set of points assigned to cluster k, you will need to compute the purity using mode($\hat{\mathbf{y}}_{\text{flat}}|\mathcal{C}_k$), which represents the most frequent greedy label in each cluster, with the detailed equation:

$$\text{Purity} = \frac{1}{M} \sum_{k=1}^{K} \max_{\ell \in \{0,\ldots,62\}} \left| \{\, m \in \mathcal{C}_k \;:\; \hat{\mathbf{y}}_{\text{flat}}[m] = \ell \,\} \right|.$$

Report the Purity, and briefly interpret the value (e.g., how closely clusters align with the model's own predictions).

[Q_{11}] **Why Cluster Logits on the Test Set?** Explain (i) what information clustering the *logit space* captures beyond simple softmax arg max decoding (e.g., grouping uncertainty shapes and confusion modes), and (ii) why restricting this analysis to the *test* split is important for a fair, post-hoc diagnostic of model behavior.

7 Bonus (Optional)

[B_2] **Alternative CNN Backbone (implement & integrate).**
Propose and justify one alternative CNN backbone to replace the baseline CNN in CRNN (include but not limited to the following: ResNet, ConvNeXt, or ViTs). Implement it and integrate into your CRNN pipeline (keep the original LSTM part).

Note: You may encounter input dimension mismatch problem for the new models. In that case, adjust the dataset code to resize the images into a correct shape.

[B_3] **Alternative Sequence Model (implement & integrate).**
Replace the bidirectional LSTM with a different sequence model (include but not limited to the following: GRU, Transformer Decoder, LSTM with asymmetric hidden sizes, or LSTM/GRU + dropout). Implement it and integrate into your CRNN (keep the original CNN part).

[B_4] **Train Your Variants.**
Train each proposed variant to reasonable convergence and report training/validation curves (loss, accuracy). Include brief notes on hyperparameters and any stabilization tricks.

[B_5] **Model Comparison Table.**
Provide a table comparing (i) Baseline CRNN, (ii) CNN-variant, (iii) RNN-variant:

- Trainable parameters
- Training time per epoch
- Final validation accuracy

[B_6] **Findings & Recommendation.**
Discuss which modification helps most and why. Reflect on complexity vs. accuracy vs. decoding behavior. Would you deploy it? Justify.

8 Programming Tips

- Carefully track `time-major` vs `batch-major` shapes for CTC.
- `input_length` should reflect post-CNN time steps T per sample.
- Start with greedy decoding; switch to beam search after the loss stabilizes.
- Save checkpoints and enable `mixed_precision` if needed.

9 Submission

Assignment submission should only be done electronically on the Canvas course site.

You should submit all codes and written answers in a single Jupyter Notebook `pa2.ipynb` with markdown cells answering the [Q_n]. You should have executed the cells that load a checkpoint and run decoding on 10 images in the section **Evaluation on test set**.

10 Grading Scheme

This programming assignment will be counted towards 15% of your final course grade. The maximum scores for different tasks are shown below:

CRNN	Code	Report
[C1] Define `CHARS`, `CHARSLABEL`, `LABEL2CHAR`	3	
[C1] Implement `_load_from_raw_files`	2	
[C1] Implement `_process_single_item`	3	
[C1] Implement `create_tf_dataset`	5	
[C2] Implement `call` and `__init__` methods in the `CRNN` class	3	
[C2] Implement `_build_cnn_backbone`	7	
[Q1] Show the steps to calculate the number of parameters in layer 4		2
[Q2] Explain the output shape of layer 6		2
[Q3] Explain the purpose of the design of `stride`		3
[Q4] Explain how each column of features corresponds to different regions in the original image		2
[Q4] Explain why the approach is effective for the text recognition task		2
[Q5] Explain the hierarchical features that each LSTM layer learns		2
[Q6] Explain the trade-off		1
[C4] Implement the `CTCLoss` class	5	
[Q6] Compare and explain the difference between CTC loss and cross-entropy loss. Explain why the former one is suitable for this case		4
[C5] Implement `greedy_decode`	2	
[Q7] Compare the three decoders		3
[C6] Implement the method `build_datasets`	2	
[C6] Implement the method `build_model`	2	
[C6] Implement the method `build_optimizer`	1	
[C6] Implement the method `build_loss`	1	
[C6] Implement the method `compute_loss`	2	
[C6] Implement the method `train_step`	4	
[C6] Implement the function `evalute`	10	
[Q8] Report the output test metrics		2
[Q8] Show 6 error cases		4
[C7] Apply gradient clipping	2	
[Q9] Compare and explain the effect of gradient clipping.		4
Total (excluding bonus): 85		

Clustering	Code	Report
[C8a] Test-only forward pass to extract per-step **logits**	4	
[C8b] Concatenate to $\mathbf{Z}_{\text{flat}}$	1	
[C8c] Compute greedy labels $\hat{\mathbf{y}}_{\text{flat}}$ via softmax argmax	1	
[C9a] Standardize logits features column-wise	1	
[C9b] Run `KMeans` with $K = 63$ (seeded) and store cluster IDs	2	
[C10a][Q10a] Silhouette: define, compute (subset if needed), and report	1	1
[C10b][Q10b] Purity: define, compute vs. greedy labels, and interpret	1	1
[Q11a] Why cluster logits beyond argmax (uncertainty/confusion modes)?		1
[Q11b] Why test-only analysis (fair post-hoc diagnostic, no leakage)?		1
Total (excluding bonus): 15		
Bonus Tasks (Optional)	Code	Report
[B1] Implement the function `prefix_beam_decode`	5	
[B1] Alternative CNN backbone implementation & integration	4	
[B2] Alternative sequence model implementation & integration	4	
[B3] Train and evaluate model variants (with curves, discussion)	2	
[B4] Model comparison table (params, time, accuracy, etc.)		3
[B5] Findings & recommendation (complexity vs. accuracy vs. decoding behavior)		2
Total Bonus: +20 (max)		

If you lose points in other parts, the bonus points can help recover your score up to the maximum (100). Any marks earned beyond 100 will not be counted.

Late submission will be accepted but with penalty.

The late penalty is deduction of one point (out of a maximum of 100 points) for every hour late after 11:59pm with no more than two days (48 hours). Being late for a fraction of an hour is considered a full hour. For example, two points will be deducted if the submission time is 01:23:34.

11 Academic Integrity

Please refer to the regulations for student conduct and academic integrity on this webpage: `https://registry.hkust.edu.hk/resource-library/academic-standards`.

While you may discuss with your classmates on general ideas about the assignment, your submission should be based on your own independent effort. In case you seek help from any person or reference source, you should state it clearly in your submission. Failure to do so is considered plagiarism which will lead to appropriate disciplinary actions.